

1 chapter7: NP 完全问题

在计算理论中，Church-Turing 论题奠定了可计算性的理论基础，它明确指出一个问题的可计算性取决于其能否在图灵机上通过有限次计算获得正确结果。这一理论框架将人类面临的所有问题划分为可计算与不可计算两大类，其中不可计算问题（或称不可解问题）具有本质上的局限性——无论投入多少计算时间与资源，都无法通过计算机获得解决方案。值得注意的是，这里的“有限次计算”是一个相对宽松的条件，主要强调**计算过程必须具有终止性**。

Cook-Karp 论题在此基础上进一步细化了可计算问题的分类标准，引入了实际可计算性的重要概念。该论题将**多项式时间**计算作为实际可计算性的判定标准，即在图灵机上通过多项式时间步数能获得正确结果的问题才被视为实际可计算的。这一划分产生了实际可计算问题与难解问题的新分类：前者具有可行的算法解决方案，后者虽然理论上仍可通过计算机求解（如采用指数时间算法），但由于缺乏多项式时间算法而被认为在实际应用中难以处理。这两个论题共同构成了计算复杂性理论的重要基石，为计算机科学中问题可解性的研究提供了理论基础。

可计算的问题需要满足：

1. 必须是存在解的问题。（例如：是否有外星生命就不行）
2. 可以通过计算机求解的问题，可以用一系列机械的数学步骤（算法）来求解。（例如：证明一个整数 n 的阶乘的末尾数只能为 0,1,4,5,6,9 种的一个（费马小定理），这个问题是有解的，但目前没有找到有效的证明方法，也不属于可计算问题）
3. 一些需要人类创造力、想象力等方面的问题，也不属于可计算问题。

算法的复杂性是指解决特定问题时，具体算法执行所需的时间量度，这反映了算法本身的效率特性。与之相对，问题的复杂性则表征问题本身固有的难度级别，这是问题与生俱来的本质属性。

从根本上看，问题的复杂度是问题自身内在的、不可改变的特性；而算法的复杂度则受到多重现实条件的制约，既包括逻辑层面的算法设计能力，也包含物理层面的实现条件限制。这种区分揭示了计算理论中问题本质属性与解决方案特性之间的重要分野。

求解 一个问题的好的算法的时复杂度函数应该恰好等于问题的计算复杂度。

算法最优性指的是在解决特定问题的所有可能算法中，效率最高的算法。这种最优性可以从两个重要维度进行衡量：最坏情况下最优和平均情况下最优。

最坏情况下最优算法是指在解决某问题的所有算法中，不存在其他算法在最坏情况下的时间复杂性比该算法更低。这意味着该算法在最不利输入情况下仍能保持最佳性能。

平均情况下最优算法则是指在解决某问题的所有算法中，不存在其他算法在平均情况下的时间复杂性比该算法更低，这表明该算法在典型输入情况下具有最优表现。这两种最优性标准为算法性能评估提供了重要理论框架。

1.1 计算复杂度的分析

1.1.1 平凡下界

有的问题必须扫描完所有输入才能得到解，算法总要得到所有输入，输出才能解决好，因此算法的输入规模/输出规模是它的平凡下界。

1.1.2 计数所需最少运算数

找最大问题：在 n 个不同的数中找最大的元素，基本元素是元素的比较，则对于任何算法在最坏情况下至少要做 $n - 1$ 次比较。

1.1.3 决策树 (Decision Tree)

决策树是一种重要的算法分析工具，它以二叉树的形式直观地展现了算法执行过程。这种特殊的树结构中，每个内部节点都代表着算法执行过程中的一次关键运算操作，比如常见的比较运算；而叶子节点则对应着算法最终输出的结果。值得注意的是，在某些特定问题中，算法的输出结果也可能出现在内部节点上，而不仅限于叶子节点。

当我们分析一个具体算法的决策树时，可以清晰地看到算法处理输入实例的完整流程。算法执行总是从决策树的根节点开始，沿着特定的路径向下遍历。在这个过程中，每个节点都会执行一次基本操作，并根据操作结果（如小于、等于或大于）决定下一步的执行路径。这种执行过程实际上就对应着从根节点到某个叶子节点或内部节点的一条完整路径，完整地记录了算法处理该输入实例的整个过程。

从算法分析的角度来看，决策树为我们提供了深入理解算法行为的窗口。对于给定的算法和固定大小的输入规模 n ，不同的输入实例会导致算法在决策树的不同节点终止。这些终止节点实际上将输入实例进行了分类，每个节点都对应着一类具有相同特征的输入。通过分析决策树中节点的总数，特别是叶子节点的数量，我们可以准确地把握算法需要处理的不同输入类别的数量，这对评估算法的复杂度具有重要意义。决策树的这种特性使其成为算法设计和分析中不可或缺的重要工具。

对于给定的 n ，任何通过比较对 n 个元素排序的算法的决策树的深度至少是 $\lceil \log n! \rceil$ 。

$$\begin{aligned}\log n! &= \sum_{j=1}^n \log j \geq \int_1^n \log x \, dx = \log e \int_1^n \ln x \, dx \\ &= \log e (n \ln n - n + 1) \\ &= n \log n - n \log e + \log e \\ &\approx n \log n - 1.5n\end{aligned}$$

存在多项式时间算法的问题是易求解的 (tractable)，不存在多项式时间算法的问题是难求解的 (intractable)。

1.2 判定问题

对于判定问题，可以使用布尔值来表示问题的答案，并通过判断输出的是“Yes”还是“No”来确定问题的解。而对于优化问题，需要确定一个最优解；而对于计数问题需要计算问题实例的数目。为了便于研究，计算复杂性理论中将问题限定为判定问题。

而其他类的问题也都可以通过适当的变换转化为判定问题，且不改变问题的难易程度，从而利用计算复杂性理论进行分析和研究。

1. 寻找多数元素问题 (MAJORITY), 给定一个含有 n 个元素的数组，寻找并返回数组中的多数元素，如果没有多数元素，则返回 none.

可以转换为，多数元素判定 (MAJORITY-D): 给定一个含有 n 个元素的数组，问数组中是否存在多数元素。

2. 哈密顿回路搜索 (HCS): 给定一个无向图 G , 搜索并返回其中一条哈密顿回路, 如果没有哈密顿回路, 则返回 "No". (哈密顿回路, 是指经过图中每个顶点恰好一次的回路, 如果图 G 中存在哈密顿回路, 则称 G 为哈密顿图。

可以转化为, 哈密顿图判定问题 (HC): 给定一个无向图 G , 判断 G 是否是哈密顿图。

3. 最长公共子序列问题 (LCS): 给定取自相同字母集合的两个序列 X 和 Y , 求 X 和 Y 的最长公共子序列。

可以转化为, K 公共子序列问题 (KCS): 给定取自相同字母集合的两个序列 X 和 Y 以及正整数 K , 问 X 和 Y 是否存在长度不小于 K 的公共子序列。

4. 货郎担问题 (TSP): 给定 n 个城市的集合, 城市 i 和城市 j 之间的距离为整数, 求一条经过每个城市恰好一次最后回到出发点的最短回路。

TSP-D: 给定 n 个城市的集合和一个正整数 K , 城市 i 和城市 j 之间的距离为整数, 问是否存在经过每个城市恰好一次最后回到出发点, 且长度不超过 K 的回路。

1.3 P 类和 NP 类

1.3.1 图灵机模型

图灵机的逻辑结构包括:

- 一个无限长度的带子, 这个带子被分成了一个一个的单元格, 每个格子有包含一个来自字母表的符号 (含有空白)。
- 一个读写头, 可在纸带上左右移动, 并可以读、修改格子上的符号。
- 一个状态寄存器, 用来存储图灵机的当前状态。图灵机的所有可能状态数目是有限的, 并且有一个特殊的状态, 称为停机状态。
- 一个指令表 (控制单元), 根据机器当前所处的状态和带子上当前的符号, 指示机器进行特定的操作。比如: 擦除或者写入一个符号、向左或者向右移动磁头。并改变状态寄存器的值, 令机器进入一个新的状态或保持状态不变。

图灵机工作时, 大体重复如下过程:

1. 读写头读出当前格子的符号
2. 根据当前状态和读到的符号找到对应的控制指令, 并执行以下三个动作:
 - (a) 读写头在格子上擦除或写入一个数字或符号
 - (b) 图灵机状态转移到一个新的状态
 - (c) 读写头向左或向右移动一格

上述过程会一直重复, 直到机器到达一个终止状态, 或者无法继续执行操作为止。图灵机的状态转移方案可以是唯一的, 也可以不是唯一的。据此, 图灵机模型可以分为: 确定性图灵机 (Deterministic Turing Machines, DTM) 和非确定性图灵机 (Nondeterministic Turing Machines, NTM)

可以用 DTM 模型描述的算法，称之为确定性算法。设 A 是求解问题 Π 的一个算法，如果在求解问题 Π 的一个实例时，在整个执行过程中，每一步都只有一种选择，则称 A 是确定性算法。因此如果对于同样的输入，实例一遍又一遍地执行，它的输出从不改变。如果 A 的时间是关于输入实例大小的多项式时间，则称 A 是多项式时间确定性算法。

非确定性图灵机 (NTM) 是 DTM 的一种推广，NTM 和 DTM 的不同之处在于，在计算的每一时刻，根据当前状态和读写头所读的符号，NTM 存在多种状态转移方案，机器可以任意选择其中一种方案继续运行，直至停机。

可以用 NTM 模型描述的算法，称之为不确定性算法。

不确定性算法的非形式化定义：

设 A 是求解问题 Π 的一个算法，如果算法 A 以如下猜测 + 验证的方式工作，称算法 A 为不确定性 (nondeterminism) 算法：

- 猜测阶段：对问题的输入实例 x 产生一个任意字符串 y ，在算法的每次运行， y 可能不同，因此猜测是以不确定的形式工作。
- 验证阶段：在这个阶段，用一个确定性算法验证两件事：首先验证猜测的 y 是否是合适的形式，若不是，则算法停下并回答 no；若是合适形式，则继续检查它是否是 x 的解，如果确实是 x 的解，则停下并回答 yes，否则停下并回答 no。

如果猜测和检查都在输入实例的多项式时间完成，则称 A 为多项式时间不确定性算法。

注意：若 A 输出 no，并不意味着不存在一个满足要求的解，也就是说，并不意味着 A 不接受该输入，因为猜测可能不正确。

1.3.2 P 类问题

在计算复杂性理论中，P 类问题是指那些判定问题，其 yes/no 解可以通过确定性算法在多项式时间内求解。具体来说，对于一个判定问题 Π ，如果存在一个非负整数 k ，使得对于输入规模为 n 的实例，能够以 $O(n^k)$ 的时间运行确定性算法得到答案，则该问题属于 P(Polynomial) 类问题。

对于一个判定问题，如果能构造出一个多项式求解算法，则可证明该问题属于 P 类。显然，所有易解问题都是 P 类问题。（例如：排序问题、不相交集问题、最短路径问题、2 着色问题、2 可满足问题）

如果对于任意问题 $\Pi \in C$ ， Π 的补也在 C 中，我们说问题类 C 在补运算下是封闭的。比如 2 着色问题的补就是不 2 可着色。P 类问题在补运算下是封闭的。

1.3.3 NP 类问题

定义 1 (NP 类问题的验证定义)：判定问题类 NP 由这样的问题 Π 组成：对于这些问题存在一个确定性算法 A ，该算法在对 Π 的一个实例展示一个断言解时，能在多项式时间内验证解的正确性。具体而言，如果断言解导致答案是 yes，就存在一种方法可以在多项式时间内验证这个解。

定义 2 (NP 类问题的非确定性计算定义)：判定问题类 NP 由这样的判定问题组成：对于它们存在着多项式时间内运行的不确定性算法。形式化地说，如果对于判定问题 Π ，存在一个非负整数 k ，对于输入规模为 n 的实例，能够以 $O(n^k)$ 的时间运行一个不确定性算法，得到 yes/no 的答案，则该判定问题 Π 是一个 NP (nondeterministic polynomial) 类问题。

NP 类问题是否对补运算封闭，即 $NP^? = co - NP$ 尚未可知。

P 类问题是易求解的，NP 类问题是易验证的，显然， $P \subseteq NP$ 目前仍然无法判断是否有或没有 $P = NP$ ，但是因为 P 在补运算下封闭，所以有 $P \subseteq co - NP$ 。

1.4 多项式时间规约和 NP 完全性

NP 完全性 (NP-complete) 是 NP 类的一个特殊子类, 这类判定问题具有以下重要特性: 如果其中任何一个问题被证明存在多项式时间的确定性算法解, 那么 NP 类中的所有问题都将能够通过多项式时间的确定性算法求解。这意味着, 若存在这样的算法, 将直接导致 $NP = P$ 的结论成立。

在计算复杂性理论中, 问题归约是一个核心概念。具体来说, 当一个问题 A 可以归约为问题 B 时, 这意味着我们可以利用解决 B 问题的方法来直接解决 A 问题, 或者说问题 A 能够被“转化”为问题 B 的形式进行处理。

从直观上看, 这种归约关系揭示了两个问题之间的复杂度比较: 若 A 可归约为 B, 则表明 B 的时间复杂度至少与 A 相当, 或者说 B 的求解难度不低于 A。换句话说, 问题 A 的难度不会超过问题 B 的难度, 因为只要能解决 B, 就必然能解决 A。

定义: 设判定问题 $\Pi_1 = \langle D_1, Y_1 \rangle$ 和 $\Pi_2 = \langle D_2, Y_2 \rangle$, 如果存在一个函数 $f: D_1 \rightarrow D_2$ 满足如下条件:

(1) f 是多项式时间可计算的, 即存在计算 f 的多项式时间算法;

(2) 对所有 $I \in D_1$, $I \in Y_1 \Leftrightarrow f(I) \in Y_2$

则称 f 是 Π_1 到 Π_2 的多项式时间归约函数。计算 f 的多项式时间算法 F 称为归约算法。

定义: 设判定问题 Π_1 和 Π_2 , 如果存在 Π_1 到 Π_2 的多项式时间归约算法, 则称 Π_1 可多项式时间归约到 Π_2 , 记作 $\Pi_1 \propto_{\text{poly}} \Pi_2$ 。

证明. 哈密顿回路问题 (HC) 可多项式归约到货郎担问题 (TSP), 即 $HC \propto_{\text{poly}} TSP$ 。

1. 问题定义:

- 设 $HC = \langle D_1, Y_1 \rangle$, 其中 D_1 为所有无向图的集合, Y_1 为包含哈密顿回路的图
- 设 $TSP = \langle D_2, Y_2 \rangle$, 其中 D_2 为所有带权完全图与界限值的组合, Y_2 为存在长度不超过 K 的周游路线的实例

2. 构造归约函数: 定义函数 $f: D_1 \rightarrow D_2$, 对任意 $I \in D_1$ (无向图 $G = \langle V, E \rangle$):

$$f(I) = (G', K)$$

其中:

- G' 为顶点集 V 上的完全图, $E' = \{(u, v) \mid u, v \in V\}$
- 边权赋值规则:

$$d(u, v) = \begin{cases} 1, & \text{若 } (u, v) \in E \\ 2, & \text{若 } (u, v) \notin E \end{cases}$$

- 界限值 $K = |V|$

显然 f 是多项式时间可计算的。

3. 等价性证明: 对任意周游路线 (经过每个顶点恰好一次):

- 包含 $|V|$ 条边, 每条边权值为 1 或 2
- 路线长度 L 满足 $|V| \leq L \leq 2|V|$

因此有：

$$\begin{aligned}
 f(I) \in Y_2 &\Leftrightarrow \exists \text{周游路线长度 } L \leq K = |V| \\
 &\Leftrightarrow L = |V| \quad (\text{因 } L \geq |V|) \\
 &\Leftrightarrow \text{路线中所有边权均为1} \\
 &\Leftrightarrow \text{该路线所有边都属于 } E \\
 &\Leftrightarrow \text{该路线是 } G \text{ 中的哈密顿回路} \\
 &\Leftrightarrow I \in Y_1
 \end{aligned}$$

综上， $I \in Y_1 \Leftrightarrow f(I) \in Y_2$ 且 f 为多项式时间可计算，故 $HC \propto_{\text{poly}} TSP$ 得证。 $\square$

定理：如果 $\Pi_1 \propto_{\text{poly}} \Pi_2$ ，则 $\Pi_2 \in P$ 蕴含 $\Pi_1 \in P$ ； Π_1 是难解的蕴含 Π_2 是难解的。

也即表明：

定理：如果 $\Pi_1 \propto_{\text{poly}} \Pi_2$ ，则相对于多项式时间，如果 Π_2 是易解的，则 Π_1 也是易解的，即 Π_2 不会比 Π_1 更容易。或者反过来讲，即 Π_1 不会比 Π_2 更难。

另外，也容易证明多项式归约具有传递性：

定理：设 Π_1 、 Π_2 和 Π_3 是三个判定问题，且有 $\Pi_1 \propto_{\text{poly}} \Pi_2$ ， $\Pi_2 \propto_{\text{poly}} \Pi_3$ ，则 $\Pi_1 \propto_{\text{poly}} \Pi_3$ 。

1.5 NP 完全性

定义：对于一个判定问题 Π ，如果对所有 $\Pi' \in NP$ ，都有 $\Pi' \propto_{\text{poly}} \Pi$ ，则称 Π 是 NP 难的 (NP-hard, NPH)。

含义：NPH 问题不会比 NP 类中的任何问题容易。

定义：对于一个判定问题 $\Pi \in NP$ ，如果对所有 $\Pi' \in NP$ ，都有 $\Pi' \propto_{\text{poly}} \Pi$ ，则称 Π 是 NP 完全的 (NP-complete, NPC)。

含义：NPC 问题是 NP 类中的 NPH 问题。由此也可以推断，NPC 问题是 NP 类中最难的问题。NP 完全性是判定 P 是否等于 NP 的关键。

NPH 问题和 NPC 问题的唯一差别是：NPC 问题必须是属于 NP 类的问题，而 NPH 问题不一定在 NP 类中。

1.6 Cook-Levin 定理

库克-列文定理。

定理 (NP 困难的传递性)：如果存在 $\Pi' \in NPH$ ，使 $\Pi' \xrightarrow{\text{poly}} \Pi$ ，则 Π 是 NPH 的。

定理 (NP 完全性的传递性)：如果 $\Pi \in NP$ ，且存在 $\Pi' \in NPC$ ，使得 $\Pi' \xrightarrow{\text{poly}} \Pi$ ，则 Π 是 NPC 的。

上述定理表明，如果要证明一个问题 Π 是属于 NPH (NPC) 的，不再需要证明所有 NP 问题都多项式时间可归约到 Π ，而是只需要证明存在一个 NPH (NPC) 的问题 Π' ， Π' 多项式时间可归约到 Π 即可。

因此，要证明一个问题 Π 是属于 NPC 的，现在只需要证明两点：(1) $\Pi \in NP$ ；(2) 存在一个 $\Pi' \in NPC$ ，使得 $\Pi' \xrightarrow{\text{poly}} \Pi$ 。

可满足性问题是第一个被证明的 NPC 问题，SAT 问题是指：是否存在一组对所有布尔变量的赋值，使得整个合取范式为真。库克列文定理表明，SAT 问题是 NPC 的。

证明一个问题 P_i 是 NPC 问题需要 2 步进行：

1. 证明问题是 NP 类问题。
2. 证明存在一个 NPC 问题 Π' ，使得其可以规约到问题 Π

已知 HC 问题是 NPC 问题，证明 TSP 问题也是 NPC 问题。

证明. (1) $TSP \in NP$

可设计如下多项式不确定性运行算法：对任意给定的一个边带权的完全无向图 G 和正整数 K ，随机猜测 G 中所有顶点的一个排列代表一条恰好经过每个城市一次的周游路线，然后检查这个排列相邻的两个顶点之间边以及排列的首尾顶点之间边的总值之和是否不超过 K 。如果是，则回答“*Yes*”，否则回答“*No*”。当 G 中存在长度不超过 K 的周游路线时，总能猜对一个顶点的排列，它确实给出一条长度不超过 K 的周游路线，从而使算法回答“*Yes*”；如果 G 中不存在长度不超过 K 的周游路线，则猜测的任何排列表示的周游路线长度都会大于等于 K ，从而算法总是回答“*No*”。显然，猜测一个排列和检查这个排列对应的周游路线的长度是否不超过 K 都可以在多项式时间内完成。因此，TSP 是 NP 类问题。任何一个构成长度不超过 K 的周游路线的顶点排列都是证据。

(2) $HC \leq_{\text{poly}} TSP$ ，前面已证。 □

1.6.1 限制法

证明 Π 包含一个已知为 NPC 问题 Π' 作为它的一种特殊情况。

最大可满足性 (Max-SAT) 问题：任给关于 n 个变元 $x_1, x_2, \dots, x_n$ 的 m 个简单析取式 $C_1, C_2, \dots, C_m$ 以及正整数 K ，问是否存在关于这些变元的赋值 t 使得 $C_1, C_2, \dots, C_m$ 中至少有 K 个为真？

对比 Max-SAT 问题和 SAT 问题，很容易看出 SAT 是 Max-SAT 问题限定 $K = m$ 的特殊情况。

当 $K = m$ 时，赋值 t 使 $C_1, C_2, \dots, C_m$ 中至少有 K 个为真就是使 $C_1, C_2, \dots, C_m$ 全部为真，这当且仅当赋值 t 使 $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$ 为真。

因此当 $K = m$ 时，Max-SAT 问题就退化为 SAT 问题。即 SAT 可以看作是 Max-SAT 的子问题。

1.6.2 局部替换法

一般来讲，如果已知 $\Pi \in NPC$ ， Π' 是 Π 在限定一定条件下的子问题，或者更一般情况， Π' 与 Π 的问题实例的构造有某方面的相似之处时，往往可以采用局部替换法来证明 Π' 的 NP 完全性。

局部替换法是选择已知 NP 完全问题的实例中的某些元素作为基本单元，然后把每个基本单元替换成指定结构，从而得到目标问题的对应实例。

证明三元可满足性问题 (3SAT) 是 NPC 问题。

显然 3SAT 问题是 SAT 问题的子问题。

对每一个 $I \in SAT$ ， I 是关于变元 $x_1, x_2, \dots, x_n$ 的合取范式 $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$ ，其中 $C_1, C_2, \dots, C_m$ 是简单析取式。

设计转换函数 f ，使得 $f(F) = F' = F'_1 \wedge F'_2 \wedge \dots \wedge F'_m$ ，其中 F'_i 是对应于 C_i 的三元合取范式，且 C_i 是可满足的当且仅当 F'_i 是可满足的。这样构造的 F' 显然满足 F 是可满足的当且仅当 F' 是可满足的。

根据 C_i 的四种可能情形，三元合取范式 F'_i 分别构造如下：

1. C_i 仅包含单个文字，设 $C_i = z_1$ ，其中 z_1 为文字，即某变元或者变元的否定（下同）。

引入两个新的变元 y_{i1} 和 y_{i2} , 令

$$F'_i = (z_1 \vee y_{i1} \vee y_{i2}) \wedge (z_1 \vee \neg y_{i1} \vee y_{i2}) \wedge (z_1 \vee y_{i1} \vee \neg y_{i2}) \wedge (z_1 \vee \neg y_{i1} \vee \neg y_{i2})$$

显然, 无论怎么赋值, $y_{i1} \vee y_{i2}$ 、 $\neg y_{i1} \vee y_{i2}$ 、 $y_{i1} \vee \neg y_{i2}$ 和 $\neg y_{i1} \vee \neg y_{i2}$ 不可能同时为真, 因此 F'_i 为真当且仅当 z_1 为真, 也即 F'_i 为真当且仅当 C_i 为真。

2. C_i 是两个文字的析取式, 设 $C_i = z_1 \vee z_2$, 其中 z_1 、 z_2 均为文字。引入一个新的变元 y_i , 令

$$F'_i = (z_1 \vee z_2 \vee y_i) \wedge (z_1 \vee z_2 \vee \neg y_i)$$

显然, 无论怎么赋值, y_i 和 $\neg y_i$ 不可能同时为真, 因此 F'_i 为真当且仅当 $z_1 \vee z_2$ 为真, 也即 F'_i 为真当且仅当 C_i 为真。

3. C_i 是三个文字的析取式, 设 $C_i = z_1 \vee z_2 \vee z_3$, 其中 z_1 、 z_2 、 z_3 均为文字。直接令 $F'_i = C_i = z_1 \vee z_2 \vee z_3$, 显然 F'_i 为真当且仅当 C_i 为真。

4. C_i 是三个以上文字的析取式, 设 $C_i = z_1 \vee z_2 \vee \dots \vee z_k$, $k \geq 4$, 其中 z_1 、 z_2 、 $\dots$ 、 z_k 均为文字。

引入 $k-3$ 个新的变元 y_{i1} , y_{i2} , $\dots$, $y_{i(k-3)}$, 令

$$F'_i = (z_1 \vee z_2 \vee y_{i1}) \wedge (\neg y_{i1} \vee z_3 \vee y_{i2}) \wedge (\neg y_{i2} \vee z_4 \vee y_{i3}) \wedge \dots \wedge (\neg y_{i(k-4)} \vee z_{k-2} \vee y_{i(k-3)}) \wedge (\neg y_{i(k-3)} \vee z_{k-1} \vee z_k)$$

设由真值指派 t 满足 C_i , 则一定存在一个 j ($1 \leq j \leq k$) 使得 $t(z_j) = 1$ 。将 t 扩展到对所有新变元的赋值上:

- (a) 如果 $j = 1$ 或者 2 时, 令 $t(y_{is}) = 0$, $1 \leq s \leq k-3$;
- (b) 如果 $j = k-1$ 或者 k 时, 令 $t(y_{is}) = 1$, $1 \leq s \leq k-3$;
- (c) 如果 $3 \leq j \leq k-2$ 时, 令

$$t(y_{is}) = \begin{cases} 1, & 1 \leq s \leq j-2 \\ 0, & j-1 \leq s \leq k-3 \end{cases}$$

不难验证, 扩展后的真值指派 t 也满足 F'_i 。因此, 如果 $t(C_i) = 1$, 则 $t(F'_i) = 1$ 。

其实证明到这里就足够了, 但是为了加强问题, 我们进一步证明等价性。

5. 反过来, 设 $t(F'_i) = 1$ 。如果 $t(y_{i1}) = 0$, 则必有 $t(z_1 \vee z_2) = 1$; 如果 $t(y_{i(k-3)}) = 1$, 则必有 $t(z_{k-1} \vee z_k) = 1$; 否则必有 s ($1 \leq s \leq k-4$), 使得 $t(y_{is}) = 1$ 且 $t(y_{i(s+1)}) = 0$, 从而 $t(z_{s+2}) = 1$ (反证可得)。因此, 如果 $t(F'_i) = 1$, 则必存在 j ($1 \leq j \leq k$) 使得 $t(z_j) = 1$, 从而如果 $t(C_i) = 1$ 。因此, F'_i 为真当且仅当 C_i 为真成立。

综合四种情况可得, F 是可满足的当且仅当 F' 是可满足的。另外, F'_i 为三元合取范式, 所包含的简单析取式的个数不会超过 C_i 中文字的 4 倍, 每个析取式由 3 个文字组成。因此, $f(F) = F'$ 可在 $|F|$ 的多项式时间内构造, 也就函数 f 是多项式时间可计算的。因此 f 是 SAT 到 3SAT 的多项式归约函数。

1.6.3 构件设计法

如果限制法和局部替换法都不起效果，可以考虑构件设计法。基本思想是用目标问题的实例设计某些构件，能够把这些构件组合起来“实现”已知的 NPC 问题实例。

对于下面 3 个问题，容易证明他们是 NP 的。下面会逐一证明 SAT 问题可以规约到他们任意之一。

- 顶点覆盖：给出一个无向图 $G = (V, E)$ 和一个正整数 k ，是否存在大小为 k 的子集 $C \subseteq V$ ，使 E 中的每条边至少和 C 中的一个顶点关联？
- 独立集：给出一个无向图 $G = (V, E)$ 和一个正整数 k ，是否存在 k 个顶点的子集 $S \subseteq V$ ，使得对于每一对顶点 $u, w \in S$ ， $(u, w) \notin E$ ？
- 团集：给出一个无向图 $G = (V, E)$ 和一个正整数 k ， G 包含一个大小为 k 的团集吗？（注意一个 G 中大小为 k 的团集是 G 中 k 个顶点的一个完全子图。）

顶点覆盖问题是 NPC 问题 给定一个有 m 个子句和 n 个布尔变元 $x_1, x_2, \dots, x_n$ 的可满足性实例 $f = C_1 \wedge C_2 \wedge \dots \wedge C_m$ ，按如下方式构造图 G ：

1. 对于 f 中的每一个布尔变元 x_i ， G 包含一对顶点 x_i 和 $\neg x_i$ ，它们有一条边相连；
2. 对于每个子句 C_j 包含的 n_j 个文字， G 包含一个大小为 n_j 的团集 C_j ；
3. 对于在 C_j 中的每个顶点 w ，有一条边连接 w 到 1 中构造的顶点对 $(x_i, \neg x_i)$ 中它相应的文字。这些边称为连通边；
4. 令 $k = n + \sum_{j=1}^m (n_j - 1)$ 。

显然，上述构造图 G 的方法可在多项式时间内完成。

引理： f 是可满足的当且仅当构造的图 G 有一个大小为 k 的顶点覆盖集。

证明：

($\Rightarrow$ 方向)

1. 考察 f 中每个文字的赋值：如果 x_i 为真，则将步骤（1）构造的对应顶点 x_i 加到顶点覆盖集中；否则，将 $\neg x_i$ 对应顶点加入到顶点覆盖集中；因此在（1）中所有边也已经被覆盖。
2. 由于 f 是可满足的，因此每个子句中至少有 1 个文字赋值为真。在步骤（2）构造的每个子句对应的团集 C_j 中与赋值为真的文字对应的顶点 u ，同（1）中对应的顶点 v 之间的连通边 (u, v) 均已经被覆盖（因为 v 已经在覆盖集中）。
3. 将每个团集 C_j 中除了顶点 u 之外的其余 $n_j - 1$ 个顶点也加入顶点覆盖集中，显然这些顶点能覆盖团集中所有边。
4. 显然，顶点覆盖集的大小为 k 。因此，如果 f 是可满足的，则图 G 有一个大小为 k 的顶点覆盖。

($\Leftarrow$ 方向)

1. 假设图 G 有一个大小为 k 的顶点覆盖集。
2. 显然在步骤（1）构造的每个边 $(x_i, \neg x_i)$ 中至少有 1 个顶点必须在覆盖集中。因此可知已有 n 个顶点在覆盖集中，同时，这些顶点为端点的连通边也由这些顶点所覆盖了。

3. 考虑剩余的 $(n_1 - 1) + (n_2 - 1) + \cdots + (n_m - 1)$ 个顶点。不难看出，对于一个大小为 n_j 的团集，顶点覆盖集的大小为 $n_j - 1$ 。因此，对每个团集，由于连通边除了与连通边有关的顶点外，其余顶点都包含在顶点覆盖中。
4. 对于 f 中的每个变元 x_i ，如果它对应的顶点在覆盖集中，则令 $x_i = \text{true}$ ，否则，令 $x_i = \text{false}$ 。这样，在每个子句对应的团集中，一定有一个顶点连接 x_i 或 $\neg x_i$ 对应赋值为真的顶点，才能使连通边被覆盖。
5. 因此，每个子句中至少有 1 个文字的赋值为真，即 f 是可满足的。

独立集问题是 NPC 问题 引理：设 $G = (V, E)$ 是连通无向图，那么 $S \subseteq V$ 是一个独立集当且仅当 $V - S$ 是 G 的一个顶点覆盖。

证明：设 $e = (u, v)$ 是 G 中的任意边：

1. S 是独立集 $\Leftrightarrow u$ 和 v 不同时在 S 中
2. $\Leftrightarrow u$ 或 v 至少有一个在 $V - S$ 中
3. $\Leftrightarrow V - S$ 覆盖了边 e
4. 由于 e 是任意边，因此 $V - S$ 是 G 的顶点覆盖

团集问题是 NPC 问题 给定一个有 m 个子句和 n 个布尔变元 $x_1, x_2, \dots, x_n$ 的可满足性实例 $f = C_1 \wedge C_2 \wedge \cdots \wedge C_m$ ，构造一个图 $G = (V, E)$ ，其中：
 $- V$ 是 $2n$ 个文字的所有出现的集合（注意：一个文字是一个布尔变元或它的否定）
 $- E = \{(x_i, x_j) | x_i \text{ 和 } x_j \text{ 位于两个不同的子句，并且 } x_i \neq \neg x_j\}$

图 G 的每个顶点对应于 f 中的每个文字，多次出现的重复表示。若图 G 中两个顶点对应的文字不互补，且不出现在同一子句中，则将其连线。（节点 $a-b$ 连线意味着文字 a 和 b 可同时为真）

引理： f 是可满足的当且仅当 G 有一个大小为 m 的团集。

证明：设 f 有 m 个子句，则有： f 可满足 $\Leftrightarrow m$ 个子句同时为真 $\Leftrightarrow$ 每个子句至少 1 个文字为真 $\Leftrightarrow G$ 中有 m 个顶点之间彼此相连 $\Leftrightarrow G$ 中有 m 个顶点的团集

1.7 本章作业

1.7.1 平面加权图的最小生成树问题的计算复杂度

设平面图 $G = (V, E)$ ， E 中的边 (i, j) 的权值为两个点 i 和 j 之间的距离。 $|V| = n$ 。计算 G 的一棵最小生成树。

证明，求解该问题的算法类的时间复杂度下界为 $\Omega(n \log n)$ 。

规约证明。将排序问题规约到最小生成树问题：

1. **输入转换**：给定待排序的 n 个实数 $\{x_1, x_2, \dots, x_n\}$ ，构造平面图 $G = (V, E)$ ：

- 顶点集 $V = \{v_1, \dots, v_n\}$ ，其中每个 v_i 对应实数 x_i
- 边集 $E = \{(v_i, v_j) | \forall i \neq j\}$ （完全图）
- 边权定义为 $w(v_i, v_j) = |x_i - x_j|$

2. **MST 的性质**：在如此构造的图中，最小生成树必定是：

$$\text{MST} = \{(v_{(1)}, v_{(2)}), (v_{(2)}, v_{(3)}), \dots, (v_{(n-1)}, v_{(n)})\}$$

其中 $v_{(i)}$ 对应排序后的第 i 小元素（即 $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$ ）。

3. **规约的正确性**：

1. 从 MST 的边可以立即读出排序结果：遍历 MST 边集即可得到有序序列
2. 该转换过程的时间复杂度为 $O(n^2)$ （构造完全图）

4. **复杂度下界**：由于比较排序问题的下界为 $\Omega(n \log n)$ ，而 MST 问题可以解排序问题，因此：

$$\text{MST 问题的复杂度下界} \geq \text{排序问题的下界} = \Omega(n \log n)$$

特别地，对于平面图，该下界仍然成立（因为规约构造的图可以是平面的）。 □

1.7.2 元素唯一性问题

设 S 是由 n 个数构成的数组，判断 S 中的元素是否唯一。如果唯一，输出“Yes”；否则输出“No”。

证明该判定问题的计算复杂度是 $\Theta(n \log n)$ 。

给定 n 维空间 E^n 中的集合：

$$W = \{(x_1, x_2, \dots, x_n) \in \mathbb{R}^n \mid x_i \neq x_j, \forall i \neq j\}$$

需先证明 W 可分解为 $n!$ 个互不相交的连通分量 W_π 。

对每个排列 $\pi \in S_n$ （ S_n 为 n 阶对称群），定义：

$$W_\pi = \{(x_1, \dots, x_n) \mid x_{\pi(1)} < x_{\pi(2)} < \dots < x_{\pi(n)}\}$$

每个 W_π 是连通的，因为：

1. 作为开凸集：由线性不等式组定义的解空间是凸的
2. 任意两点可通过直线路径连接： $\forall \mathbf{a}, \mathbf{b} \in W_\pi$ ，有 $t\mathbf{a} + (1-t)\mathbf{b} \in W_\pi$ （ $0 \leq t \leq 1$ ）

对任意两个不同排列 $\pi \neq \sigma$ ，必有：

$$W_\pi \cap W_\sigma = \emptyset$$

因为：

- 存在至少一对指标 i, j 使得 $\pi^{-1}(i) < \pi^{-1}(j)$ 但 $\sigma^{-1}(i) > \sigma^{-1}(j)$
- 在 W_π 中要求 $x_i < x_j$ ，而在 W_σ 中要求 $x_i > x_j$

所有 W_π 的并集等于 W ：

$$\bigcup_{\pi \in S_n} W_\pi = W$$

因为：

- 对任意 $\mathbf{x} \in W$ ，可通过排序得到唯一排列 π 使 $x_{\pi(1)} < \dots < x_{\pi(n)}$

- 故 $\mathbf{x} \in W_\pi$ 且不属于其他 W_σ

在 $\mathbb{R}^n$ 空间中:

- 每个超平面 $x_i = x_j$ 将空间分成两个半空间
- W_π 对应所有 $\binom{n}{2}$ 个不等式 $x_{\pi(i)} < x_{\pi(j)}$ ($i < j$) 的交集
- 这些不等式定义的区域是 $\mathbb{R}^n$ 中的开凸多面体
- 不同的严格全序排列共有 $n!$ 种
- 每个排列对应唯一的坐标顺序关系
- 因此连通分量数量为 $|S_n| = n!$

集合 W 被超平面 $\{x_i = x_j\}_{i \neq j}$ 划分为 $n!$ 个:

- 互不相交的
- 连通的
- 开凸的

分量, 每个分量对应一个唯一的排列顺序。

1.7.3 2SAT 问题

设计一个多项式时间确定算法求解 2 可满足性问题, 从而证明 2 可满足性问题是 P 类问题。

1. 构建蕴含图:

- 对于每个变量 x , 创建两个顶点: x 和 $\neg x$ 。
- 对于每个子句 $(a \vee b)$, 添加两条有向边:

$$- \neg a \rightarrow b$$

$$- \neg b \rightarrow a$$

2. 计算强连通分量 (SCC):

- 使用 Kosaraju 算法或 Tarjan 算法找到图中的所有强连通分量。
- 强连通分量是指图中任意两个顶点相互可达的最大子图。

3. 检查可满足性:

- 对于每个变量 x , 检查 x 和 $\neg x$ 是否在同一个 SCC 中。
- 如果存在这样的 x , 则公式不可满足, 返回 “不可满足”。
- 否则, 公式可满足, 返回 “可满足”。

4. 构造满足的赋值 (可选):

- 对 SCC 进行拓扑排序, 得到顺序 $S_1, S_2, \dots, S_k$ 。

- 初始化所有变量为未赋值。
- 按 $i = k$ downto 1 的顺序处理每个 SCC：
 - 如果 S_i 中的任何文字未被赋值：
 - * 将 S_i 中的所有文字赋值为真。
 - * 将它们的否定赋值为假。
- 返回得到的赋值。

1.7.4 2 着色问题

设计一个多项式时间的确定算法求解 2 着色问题，从而证明 2 着色问题是 P 类问题。

Input: 无向图 $G = (V, E)$
Output: 若 G 是二分图返回着色方案，否则返回”不可着色”

```

1 Algorithm TWO-COLORING( $G$ )
2 foreach  $v \in V$  do
3   |  $color[v] \leftarrow$  未定义
4 end
5  $queue \leftarrow$  空队列
6 任选顶点  $s \in V$ 
7  $color[s] \leftarrow$  红色
8  $queue.enqueue(s)$ 
9 while  $queue$  非空 do
10  |  $u \leftarrow queue.dequeue()$ 
11  | foreach  $v \in Adj[u]$  do
12  |   | if  $color[v] =$  未定义 then
13  |   |   |  $color[v] \leftarrow$  与 $color[u]$ 相反的颜色
14  |   |   |  $queue.enqueue(v)$ 
15  |   | end
16  |   | else if  $color[v] = color[u]$  then
17  |   |   | return ”不可着色”
18  |   | end
19  | end
20 end
21 return  $color$ 

```

Algorithm 2: 2-着色问题的 BFS 解法

正确性证明 算法基于以下观察：

- 图是二分图当且仅当可以用两种颜色着色且相邻节点颜色不同
- 使用 BFS 遍历时，若发现相邻节点颜色相同则立即终止
- 颜色冲突检测保证了算法的正确性

时间复杂度分析

- 初始化阶段: $O(|V|)$
- BFS 遍历: $O(|V| + |E|)$
- 总时间复杂度: $O(|V| + |E|)$

由于:

- 算法在多项式时间 $O(|V| + |E|)$ 内终止
- 使用确定性图遍历 (BFS/DFS)
- 不需要猜测和验证步骤

因此 2-着色问题属于 P 类问题。

1.7.5 规约问题的传递性证明

对判定问题 Π_1 和 Π_2 , 证明如下结论:

如果 $\Pi_1 \propto_{poly} \Pi_2$, 则 $\Pi_2 \in P$ 蕴含 $\Pi_1 \in P$; Π_1 是难解的蕴含 Π_2 是难解的。

证明. 设 Π_1 和 Π_2 为形式语言, 证明分为两部分:

第一部分: 正向蕴含

1. 已知 $\Pi_1 \propto_{poly} \Pi_2$, 即存在多项式时间可计算函数 f , 使得

$$\forall x \in \Sigma^*, x \in \Pi_1 \Leftrightarrow f(x) \in \Pi_2$$

2. 设 $\Pi_2 \in P$, 则存在多项式时间算法 A_2 判定 Π_2
3. 构造 Π_1 的判定算法 A_1 :
 - 输入 x , 计算 $f(x)$ (耗时 $p(|x|)$, p 为多项式)
 - 运行 $A_2(f(x))$ (耗时 $q(|f(x)|)$, q 为多项式)
 - 输出 A_2 的结果
4. 总时间复杂度为 $p(|x|) + q(|f(x)|)$, 仍为多项式时间

第二部分: 逆否命题

1. 原命题的逆否命题为: 若 $\Pi_2 \notin$ 难解类, 则 $\Pi_1 \notin$ 难解类
2. 假设 Π_2 不是难解的 (即 $\Pi_2 \in P$)
3. 由第一部分结论直接可得 $\Pi_1 \in P$, 故 Π_1 不是难解的
4. 特别地, 当“难解类”取 P 的补集时:

$$\Pi_2 \notin \bar{P} \Rightarrow \Pi_1 \notin \bar{P}$$

即 $\Pi_2 \in P \Rightarrow \Pi_1 \in P$

综上得证: 多项式时间规约具有保持问题复杂度的性质。 □

1.7.6 集合覆盖问题是 NPC 问题

集合覆盖问题：给出集合 $S = s_1, s_2, \dots, s_n$ ，和它的子集簇 $C = c_1, c_2, \dots, c_m$ ，其中 $\forall c_i, c_j \subseteq S$ ，给定正整数 k ，是否存在 C 的大小不超过 k 的子集 C' ， C' 中的集合的并是 S ？

顶点覆盖问题：给出一个无向图 $G = (V, E)$ ，和一个正整数 k ，是否存在大小不超过 k 的顶点子集 $C \subseteq V$ ，使 E 中的每条边至少和 C 中的一个顶点关联？

请证明：顶点覆盖问题 $\leq_{poly}$ 集合覆盖问题。

通过将顶点覆盖问题（Vertex Cover）规约到集合覆盖问题（Set Cover），证明集合覆盖问题属于 NPC 类。

规约构造 给定顶点覆盖实例 $\langle G = (V, E), k \rangle$ ，构造集合覆盖实例 $\langle S, C, k' \rangle$ 如下：

1. 令全集 $S = E$ ，即所有边构成的集合
2. 对每个顶点 $v \in V$ ，创建子集 $s_v = \{e \in E \mid \text{边 } e \text{ 与顶点 } v \text{ 关联}\}$ ，构成子集簇 $C = \{s_v \mid v \in V\}$
3. 保持参数不变，设 $k' = k$

正确性证明

1. 正向证明（Vertex Cover $\Rightarrow$ Set Cover）：

- 设存在大小为 $j \leq k$ 的顶点覆盖 $V' = \{v_1, \dots, v_j\}$
- 构造对应的子集簇 $C' = \{s_{v_1}, \dots, s_{v_j}\}$
- 对任意边 $e = (u, w) \in E$ ：
 - 由顶点覆盖性质， u 或 w 至少一个在 V' 中
 - 若 $u \in V'$ ，则 $e \in s_u \in C'$
- 故 C' 覆盖 S ，且 $|C'| = j \leq k$

2. 反向证明（Set Cover $\Rightarrow$ Vertex Cover）：

- 设存在大小为 $j \leq k$ 的集合覆盖 $C' = \{s_{v_1}, \dots, s_{v_j}\}$
- 构造对应的顶点集 $V' = \{v_1, \dots, v_j\}$
- 对任意边 $e = (u, w) \in E$ ：
 - 存在 $s_v \in C'$ 包含 e
 - 故 e 至少与 $v \in V'$ 关联
- 故 V' 是顶点覆盖，且 $|V'| = j \leq k$

3. 时间复杂度分析：

- 构造全集 S ： $O(|E|)$
- 构建子集簇 C ： $O(|V| \cdot \deg(v)) = O(|V||E|)$
- 总时间复杂度为多项式时间 $O(|V||E|)$

结论 通过上述多项式时间规约，且已知顶点覆盖问题是 NP 完全问题，可证明集合覆盖问题属于 NPC 类。

1.7.7 0-1 背包问题

0-1 背包问题的动态规划算法是一个多项式时间算法吗？解释你的答案。

0-1 背包问题的动态规划算法不是一个多项式时间算法。该算法的运行时间为 $O(nW)$ ，其中 n 是商品数量， W 是背包能容纳的最大商品总重量。 W 的二进制编码长度为 $l = \lceil \lg W \rceil$ ，该算法的运行时间为 $O(n2^l)$ ，随着 l 的增加， $O(n2^l)$ 呈指数级增长。我们通常称 0-1 背包问题为一个伪多项式时间算法⁺(pseudo-polynomial time algorithm)。

1.7.8 子例程时间影响

证明：若一个算法对多项式时间子例程的调用次数为常数，并且每次执行的额外工作量也需要多项式时间，则该算法的运行需要多项式时间。若多项式时间子程序的调用次数修改为多项式，则该算法的运行可能需要指数时间。

首先证明第一个命题，假设输入规模为 n ，调用 c 次多项式时间子例程，其中 c 为常数，因为每次迭代的输出输入为下一次迭代的输入。设第 i 次迭代多项式时间子例程的运行时间为 $O(n^{k_i})$ ，其中 k_i 为常数，多项式时间的额外工作量为 $O(n^{l_i})$ ，其中 l_i 为常数，第一次迭代的运行时间为 $T_1(n) = O(n^{k_1}) + O(n^{l_1}) = O(n^{\max\{k_1, l_1\}})$ 为多项式时间，输出规模为 $n_1 = O(T_1(n)) = O(n^{\max\{k_1, l_1\}})$ ，第二次迭代的运行时间为

$$T_2(n) = O(n_1^{\max\{k_2, l_2\}}) = O((n^{\max\{k_1, l_1\}})^{\max\{k_2, l_2\}}) = O(n^{\max\{k_1, l_1\} \max\{k_2, l_2\}})$$

为多项式时间，输出规模为 $n_2 = O(T_2(n)) = O(n^{\max\{k_1, l_1\} \max\{k_2, l_2\}})$ ，采用归纳法，第 i 次迭代的运行时间 $T_i(n) = O(n^{\prod_{j=1}^i \max\{k_j, l_j\}})$ 为多项式时间，输出规模为

$$n_i = O(T_i(n)) = O(n^{\prod_{j=1}^i \max\{k_j, l_j\}})$$

总的运行时间为

$$\sum_{i=1}^c T_i(n) \leq c \max\{T_1(n), T_2(n), \dots, T_c(n)\}$$

为多项式时间。

然后证明第二个命题，举例如下，假设开始输入规模为 1，每次迭代输出规模为输入规模的两倍， n 次迭代后输出规模为 2^n ，输出规模与运行时间多项式相关，该算法的运行时间为指数时间。证明完毕。

Input: 有向图 $G = (V, E)$

Output: 强连通分量集合 $SCCs$

```
1 Algorithm TARJAN( $G$ )
2  $index \leftarrow 0$ 
3  $S \leftarrow \emptyset$ 
4 foreach  $v \in V$  do
5    $index[v] \leftarrow$  未定义
6    $low[v] \leftarrow$  未定义
7    $onStack[v] \leftarrow$  false
8 end
9  $SCCs \leftarrow \emptyset$ 
10 foreach  $v \in V$  do
11   if  $index[v] =$  未定义 then
12     STRONGCONNECT( $v$ )
13   end
14 end
15 return ( $SCCs$ )

16 Function STRONGCONNECT( $v$ )
17  $index[v] \leftarrow index$ 
18  $low[v] \leftarrow index$ 
19  $index \leftarrow index + 1$ 
20  $S.push(v)$ 
21  $onStack[v] \leftarrow$  true
22 foreach  $(v, w) \in E$  do
23   if  $index[w] =$  未定义 then
24     STRONGCONNECT( $w$ )
25      $low[v] \leftarrow \min(low[v], low[w])$ 
26   end
27   else if  $onStack[w]$  then
28      $low[v] \leftarrow \min(low[v], index[w])$ 
29   end
30 end
31 if  $low[v] = index[v]$  then
32    $C \leftarrow \emptyset$ 
33   repeat
34      $w \leftarrow S.pop()$ 
35      $onStack[w] \leftarrow$  false
36      $C \leftarrow C \cup \{w\}$ 
37   until  $w = v$ 
38    $SCCs \leftarrow SCCs \cup \{C\}$ 
39 end
```

Algorithm 1: Tarjan 算法求解强连通分量